

NOVEMBER 19, 2024



DATA VISUALISATION FOR FOOTBALL STATISTICS

BIG DATA FUNDAMENTALS COURSE WORK

YAZMIN CACERES FEGRERO

ZAINAB ALKANDARI

2.735 WORDS

Table of Contents

LIST OF FIGURES AND TABLES	1
INTRODUCTION	2
DATASET: PREMIER CHAMPION LEAGUE	3
2.1: GOAL AND EMPHASIS	3
2.2 OBJECTIVE	3
2.3 SOURCE	3
UNSUPERVISED ANALYSIS - CLUSTERING	12
AGGLOMERATIVE CLUSTERING	13
K-MEANS	14
SUPERVISED ANALYSIS	ERROR! BOOKMARK NOT DEFINED.
LOGISTIC REGRESSION	16
<i>Integration with Clustering Results</i>	16
<i>Confusion Matrix Summary</i>	16
<i>Performance Metrics Summary</i>	17
OPTIMISATION OF MODELS	19
LIMITATIONS	20
REFLECTIONS	20
CONCLUSION	21
REFERENCES	22
APPENDIX	23

List of Figures and Tables

<i>Figure 1: Total Score by Team in Premier League (2019-2020)</i>	5
<i>Figure 2: Distribution of Key Performance Metrics in Premier League Matches (2019-2020)</i>	5
<i>Figure 3: Log-Transformed Mean, Median, and Mode of Key Performance Metrics in Premier League Matches (2019-2020)</i>	6
<i>Figure 5: Average Performance Heatmap by Team The heatmap identifies team strengths and allows performance comparison. High possession values are shown by intensified hues. Colours with reduced saturation indicate low values. This quickly identifies teams with high or low metric values. A lighter foul colour implies fewer team offences. Liverpool has the fewest fouls (9.82) and Watford the most (15.16).</i>	9
<i>Figure 6: Score Distribution Across Classes in the Dataset</i>	9
<i>Figure 7: Average Scores of Top 5 Premier League Teams (2019-2020)</i>	10
<i>Figure 8: Heatmap of Correlation Between Key Football Performance Metrics</i>	11
<i>Figure 9: Clustering Evaluation Scores for Agglomerative Methods</i>	14
<i>Figure 10: Evaluation of K-Means Clustering Performance (K=2 to 50) on Premier League Data</i>	15
<i>Figure 11: Comparison of Metrics Between Models</i>	18
<i>Figure 12: Clusters Generated by K-Means Algorithm</i>	19

Tables

<i>Table 1: Premier League Match Performance Data</i>	3
<i>Table 2: Confusion Matrix of Predicted vs. Actual Classes</i>	16
<i>Table 3: Classification Report Summary with Metrics for Precision, Recall, F1-Score, and Support</i>	17

Introduction

Established in 1992, the Premier Competition is the premier football competition in England and is regarded as one of the most popular and competitive leagues worldwide. The teams play 38 games in a season. The league is renowned for drawing the top coaches and players because of its intense competition (Premier League, n.d.).

Finding performance trends, evaluating each team's efficacy, and examining trends in cumulative scores are the goals of this approach.

By examining these trends, teams and management can enhance team performance, boost fan engagement, foster innovation in sports technology, and make well-informed decisions.

The accuracy of the data is one of the problems with football data because it may be manually gathered.

Dataset: Premier Champion League

Goal and emphasis

The goal is to evaluate Premier League football data for the 2019–2020 season. By comparing teams' possession, shooting, and accuracy statistics, we may look for trends in match performance. This will assist in determining the main elements affecting team tactics and match results.

Objective

Our goal is to inspire sports statistics firms to invest in data visualisation and dashboards by presenting football data in a way that is interesting and simple for managers, coaches, players, and fans to understand.

Source

All the data used was publicly obtained from Kaggle.com; however, some changes were necessary to obtain the final dataset, including cleaning and removing irrelevant columns that had only null values. (Penalties, PassingAccuracy). All data is used solely for the season of 2019/2020 of the premier league.

Table 1: Premier League Match Performance Data

Category	Column	Description
Match Information	Match_Name	Names of teams involved in each match
	Match_Date	Date the match took place
Performance Stats	Score	Total goals scored by the team
	Fouls	Total fouls committed
	Corners	Total corners taken

Possession and Passing	Possession	Possession Percentage
	SucPass	Successful passes completed
	NumofPass	Total number of passes attempted
Shooting Accuracy	ShotsAccuracy	Shot accuracy percentage
	SucShots	Number of successful shots Shots on target
	NumofShots	Total shots taken
Goalkeeping Stats	SavesAccuracy	Save accuracy percentage
	SucSaves	Successful saves
	NumofSaves	Total saves attempted

Preliminary analysis presents trends in scoring, ball control, and team performance across variables, providing insight into the data set (2.3). These results lay the framework for more advanced statistical and machine-learning tools to better comprehend team tactics and production. 2.3.1 Team total score. The dataset includes each team's 2019-2020 Premier League points. Importantly, each Premier League side can earn points based on match results:

- Winner gets 3 points.
- Tie = 1 point.
- Loss = zero points.

A team's maximum score relies on the season's matches. Teams play 38 games per season, with a maximum score of 114 points (3 points per win).

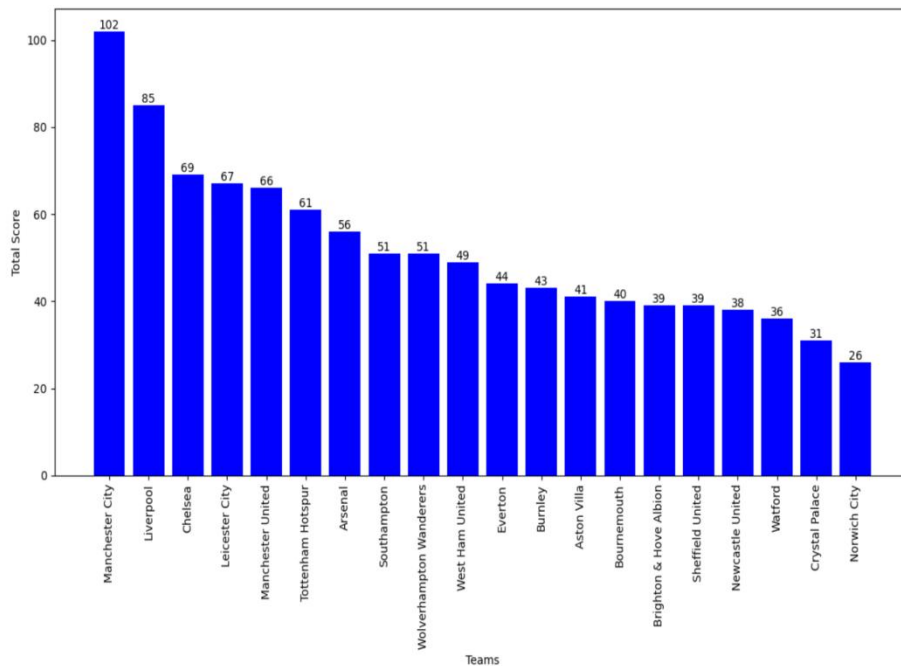


Figure 1: Total Score by Team in Premier League (2019-2020)

The bar chart shows each team's season points for performance comparisons. Manchester City, known for their stability and ability to score, lead with 102 points. Liverpool and other great teams follow. Teams like Norwich City and Crystal Palace are at the bottom of the list, showing they are struggling to score.

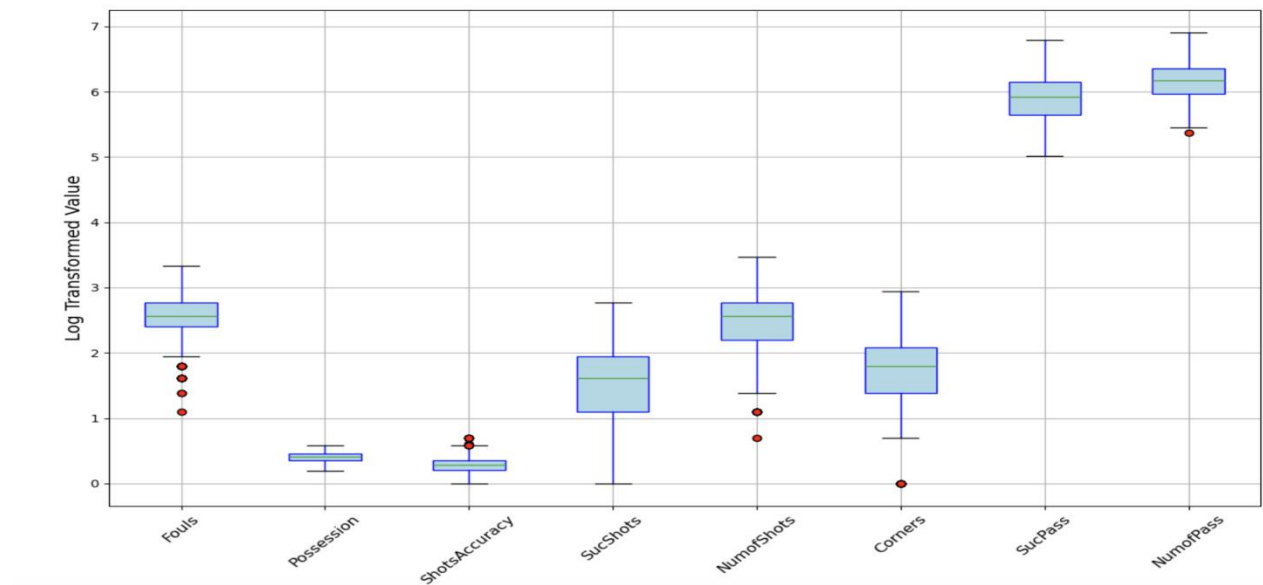


Figure 2: Distribution of Key Performance Metrics in Premier League Matches (2019-2020)

In the box figure above, the whiskers reflect the range of values excluding outliers, and each box shows the IQR for each dataset column. The median line is inside each box. Red dots are outliers. Log-formed values reduce extreme outliers and enhance distribution readability.

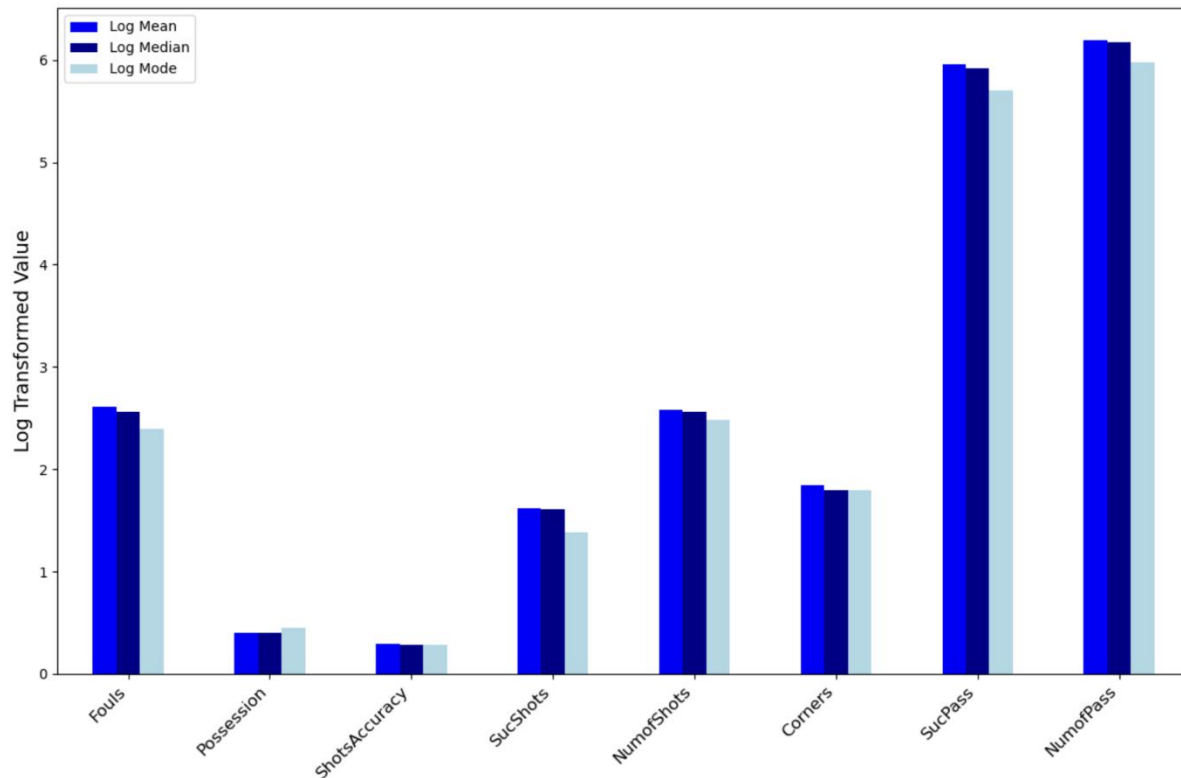


Figure 3: Log-Transformed Mean, Median, and Mode of Key Performance Metrics in Premier League Matches (2019-2020)

The bar chart above illustrates the trends in important metrics in 2019/2022 Premier League matches. The log transformation mitigated excessive outliers. mean—average. median—central tendency. Mode—most common. We compare these measurements to see skewness and metric distributions.

Possession Distribution: manchester city vs. liverpool on 2020-07-02

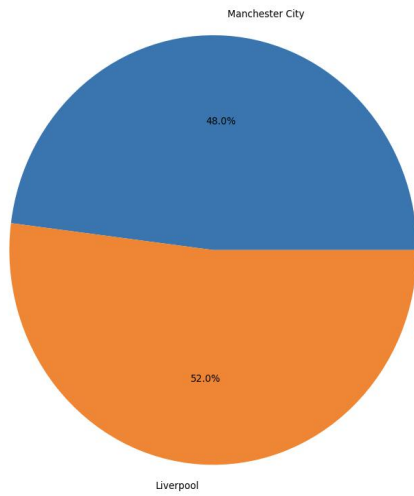


Figure 2.3.3: Team Possession Comparison in Premier League (2019-2020)

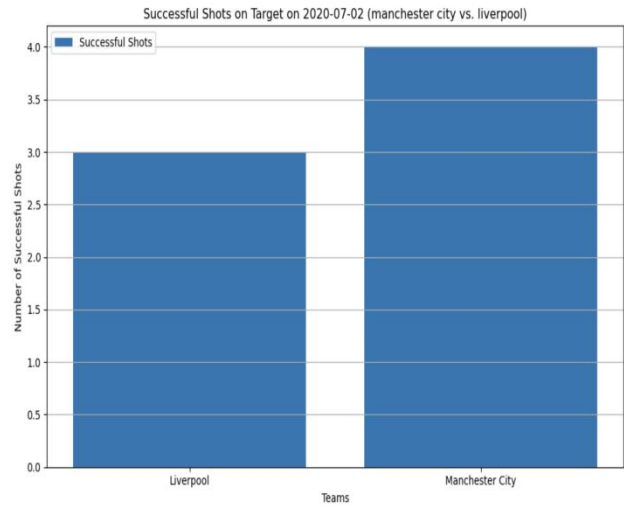


Figure 2.3.4: Shots on Target Distribution Between manchester city and liverpool in one game on 2020-07-02.

In a Premier League 2019/2022 match, the graphs above may show possession or SucShots for each team. Date-based game selection. We created an easy query to present match statistics, showing possession with a pie chart and successful shots with stacked bars. Appendix A provides the interactive user query for match statistics.

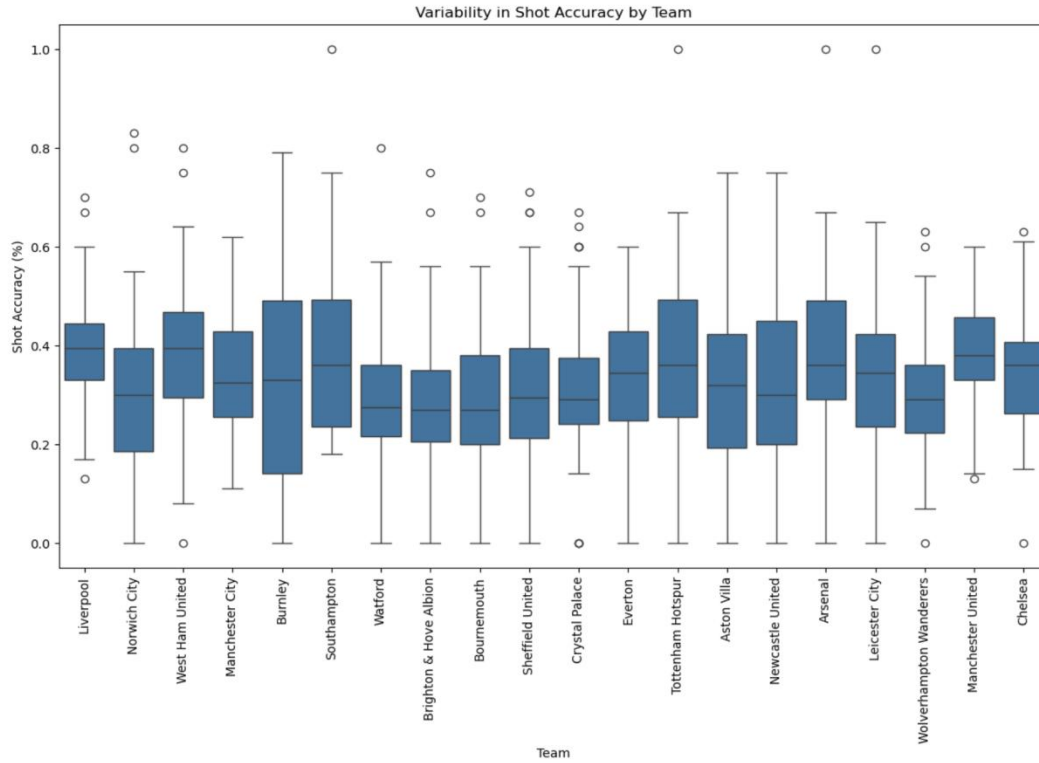


Figure 4: Variability in Shot Accuracy by Teams

The objective of the graphic above is to evaluate the variation of specific key metrics for each team and ascertain whether their performance is consistent or fluctuates from game to game. Teams with a high median exhibit superior shooting accuracy, but those with a low median or extensive range may demonstrate diminished accuracy and potential for enhancement. Points outside the range indicate games with very high or low shooting accuracy, warranting analysis.

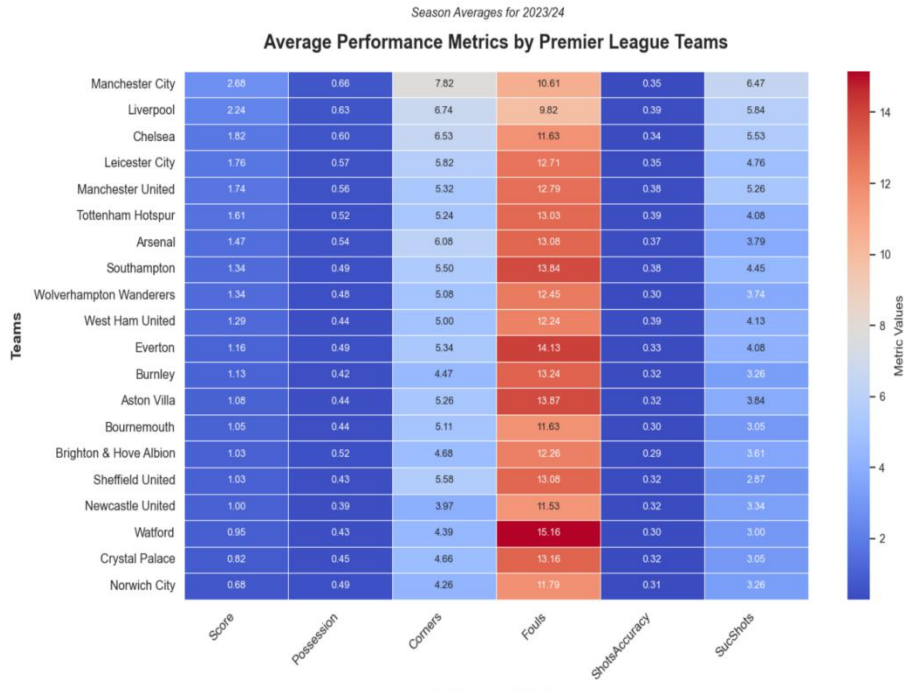


Figure 5: Average Performance Heatmap by Team

The heatmap identifies team strengths and allows performance comparison. High possession values are shown by intensified hues. Colors with reduced saturation indicate low values. This quickly identifies teams with high or low metric values. A lighter foul color implies fewer team offences. Liverpool has the fewest fouls (9.82) and Watford the most (15.16).

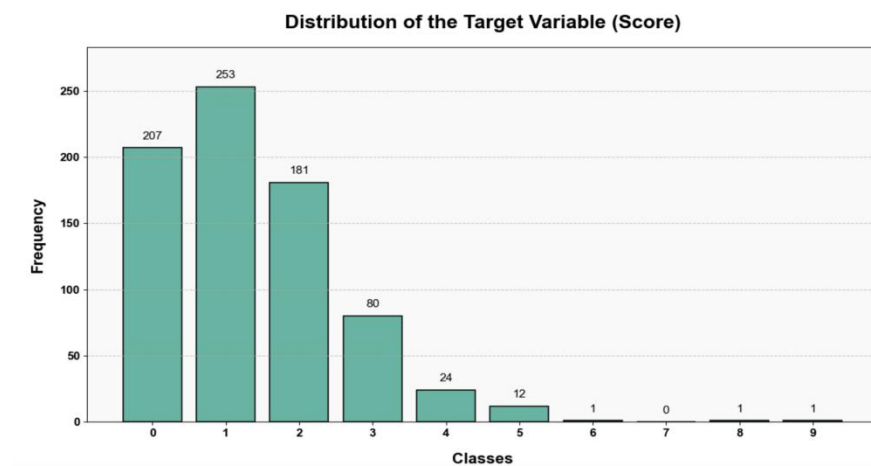


Figure 6: Score Distribution Across Classes in the Dataset

This graph shows the distribution of Score, the amount of goals each side scored in documented matches. The X-axis shows goal scores, while the Y-axis shows dataset score frequency.

In football matches, 0–1 scores are common. This distribution is skewed, with many low scores and few high numbers. This must be considered when creating the supervised model.

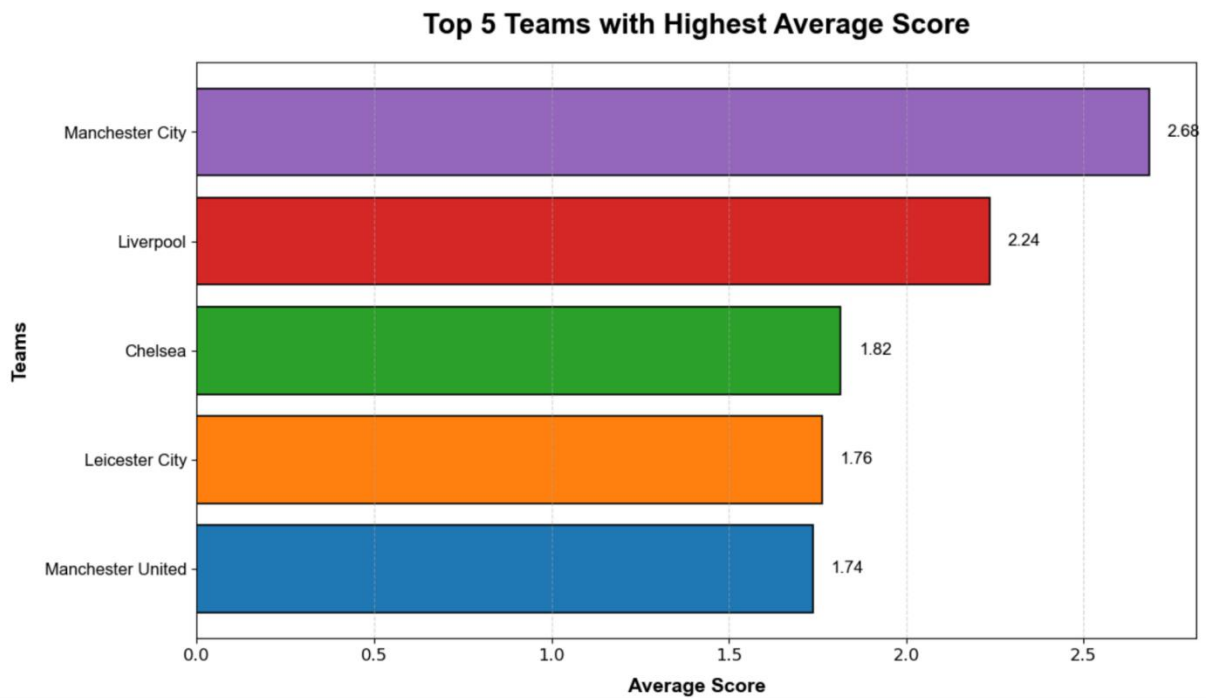


Figure 7: Average Scores of Top 5 Premier League Teams (2019-2020)

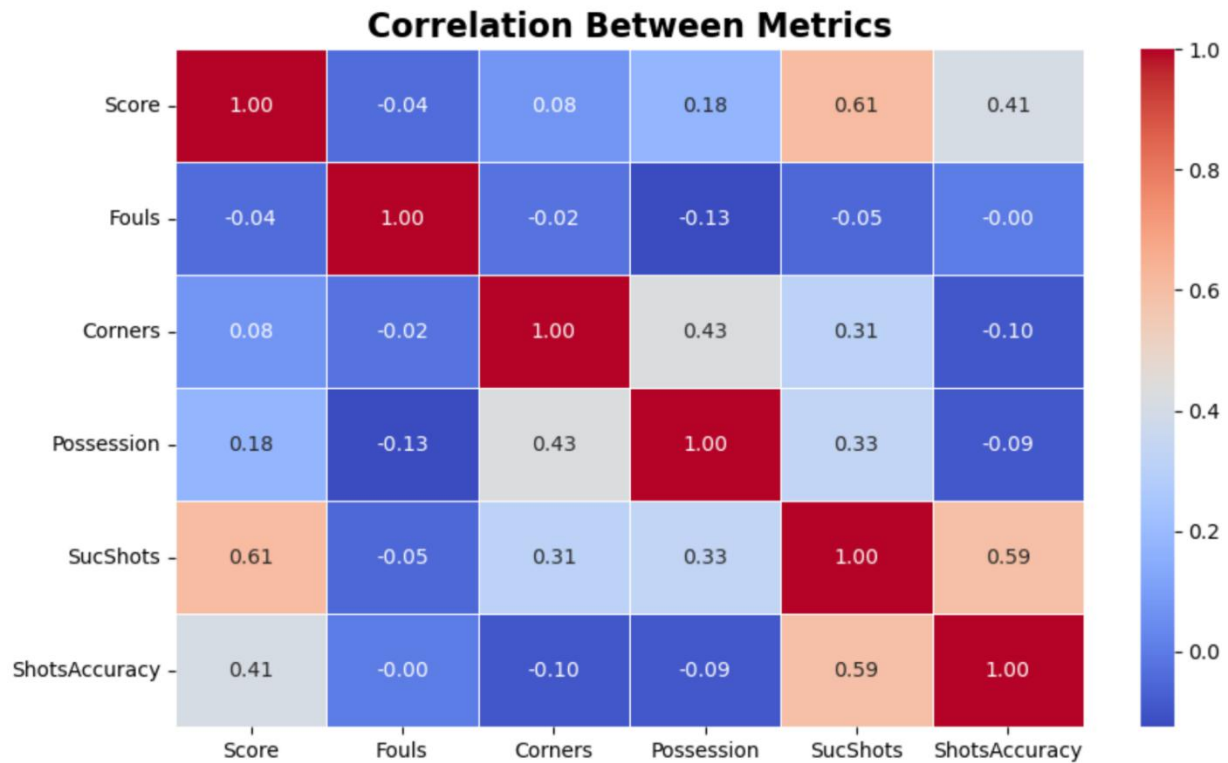


Figure 8: Heatmap of Correlation Between Key Football Performance Metrics

The heat map illustrates relationships among football performance indicators. Strong positive correlations are shown between SucShots (successful shots) and Score (goals) at 0.61, as well as between ShotsAccuracy and SucShots at 0.59. These positive relationships indicate the influence of these measurements on goal scoring. The indicators exhibiting weak or negligible correlation, such as fouls, have a minimal influence on goal scoring. The examination of performance through heat map correlation may assist in prioritising the influence of measurements on team performance, hence enhancing overall performance.

Unsupervised analysis - Clustering

Unsupervised analysis

An unsupervised analysis was performed to identify concealed patterns and clusters within our dataset. Our study largely concentrates on team performance and match statistics. Our objective is to identify analogous team performances and uncover match patterns to comprehend the aspects influencing outcomes. Nevertheless, our dataset is not well configured for unsupervised analysis, as the performance indicators may exhibit excessive similarity between teams, complicating the differentiation of patterns. Furthermore, our dataset includes columns measured on a continuous scale, such as possession, shot accuracy, and score. We address this by employing binning into intervals, categorising certain ranges of each feature into intervals, grouping analogous values, and assigning them ratings of low, medium, and high.

Agglomerative Clustering

Agglomerative analysis is a method of hierarchical clusters, the data points in this method are grouped based on their similarities. The football dataset is not very ideal for unsupervised analysis because of similarities across teams. So we use a more comprehensive analysis, using Scikit-Learn. The closeness of data we determine by calculating the distances with an n-dimensional space. 4 linkage methods show how the clustering algorithm links those clusters together.

- **Single:** it merges cluster based on closest points between clusters
- **Average:** it merge clusters based on average distance between the points in cluster
- **Complete:** it merge the clusters that are most far from each other
- **Ward:** it merge the clusters that add least spread when combined

The evaluation metrics used to calculate the difference between clusters differently.

Silhouette Score: It tells how well-separated the clusters are. Range (from +1 to Negative) +1 – clusters are very well separated, 0 – not clearly separated and negative – clusters are overlapping, not well formed.

Completeness Score: it tells how similar things are grouped together in the same cluster, a (range 0 to 1), where 1 – all similar data points in the same cluster, 0 – similar points are spread out across different clusters.

Homogeneity Score: it tells how pure the clusters are, meaning each cluster has similar items, (range is 0 to 1). 1 -- each cluster has only one type of data, 0 -- clusters are mixed with different types of data.

The selection of these methods is to explore the clustering analysis to find where patterns exist in football data. From (Figure 3.1) Ward and complete methods may struggle in grouping similar points and when creating pure clusters. The average linkage performs differently than the ward and complete linkage,

it separates clusters better. The least preferred method would be single. The distance measurements are Euclidean, Manhattan and Cosine because the performance of clustering depends on them. they appear to have a greater influence on the Silhouette Score than the linkage method. Cosine has a stronger effect on separating the clusters. Euclidean works better when it comes to grouping similar points and when forming pure clusters. Therefore, the distance measurements seem to have a greater influence on results in our football dataset than the algorithm itself.

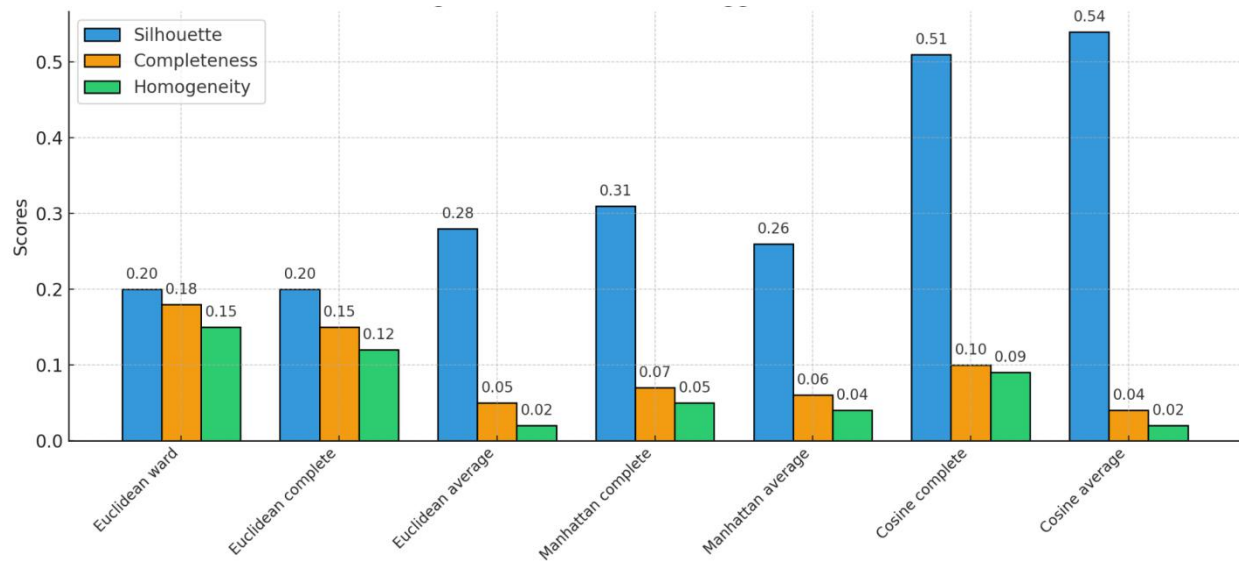


Figure 9: Clustering Evaluation Scores for Agglomerative Methods

K-Means

K-means clusters data and the elbow technique determines the best number of clusters. The elbow method's blue line in Figure 3.2 shows inertia for each K (2–5 clusters). After a dramatic decrease in inertia, the pace slows down, indicating that the elbow is about $K = 10$, which is the optimal number of clusters. Even with $K = 10$, the Silhouette score (Green line) demonstrates that the separation is not optimum, maybe due to poorly defined clusters or insufficient data. As K grows, the Completeness Score (Orange line) shows that the clustering algorithm is moderately good at grouping related points, but it is not flawless. Last is the Homogeneity Score (Red line), which indicates clusters contain distinct data types or classes. In conclusion, $K=10$ is best based on inertia, however Silhouette, Completeness, and Homogeneity scores show the clusters may be too similar.

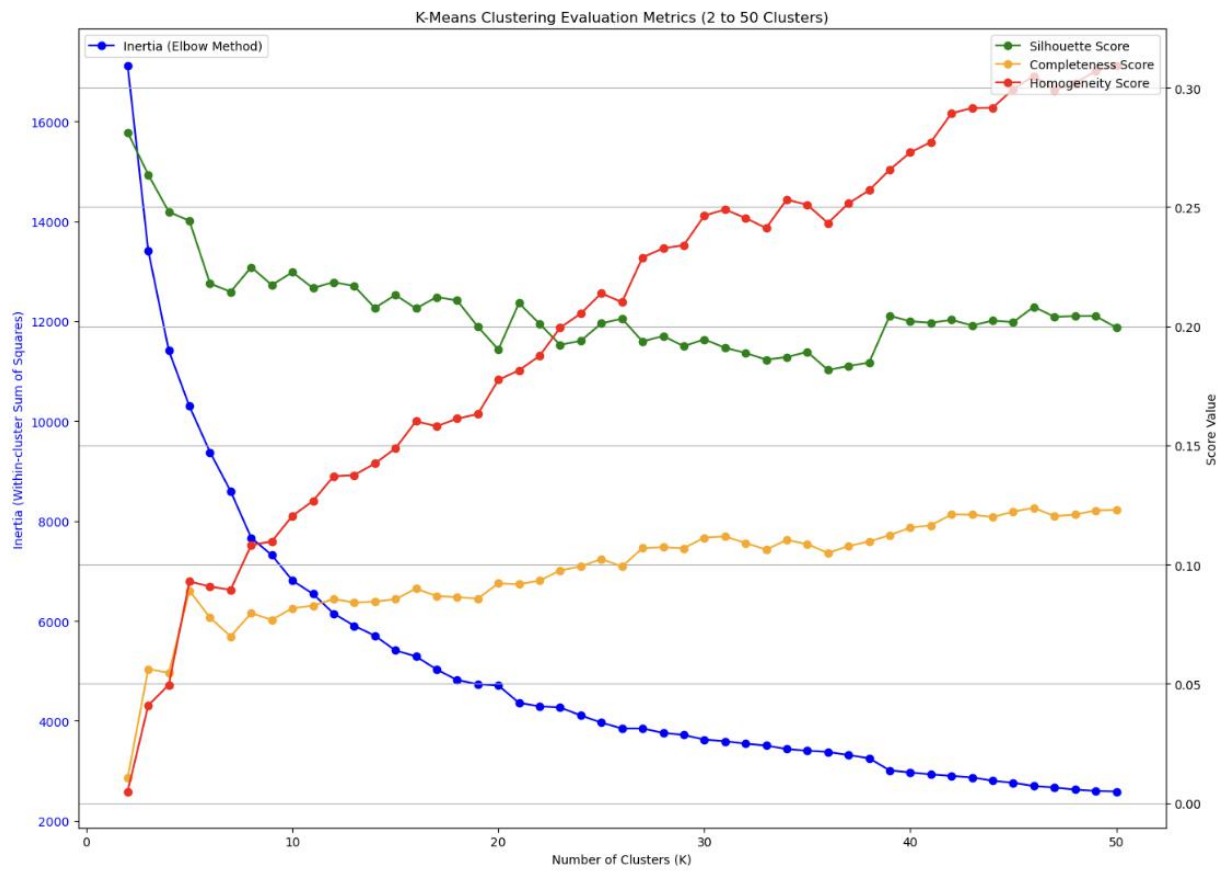


Figure 10: Evaluation of K-Means Clustering Performance (K=2 to 50) on Premier League Data

Supervised analysis

Supervised methods train models to classify or predict outcomes from labelled data (Mitchell, 1997). In this analysis, we used Premier League performance metrics such as possession and shots to predict key indicators such as score and evaluate team performance.

Two models were used: Logistic Regression, which allows the impact of each metric to be interpreted using coefficients, and Decision Tree, a robust model that does not require data scaling. To ensure the model's validity and avoid overfitting, 30% of the data was reserved for testing, following good practices (Géron, 2019). Performance was assessed using precision, recall, and F1-Score metrics, highlighting strengths and limitations in an unbalanced dataset.

Logistic regression

Integration with Clustering Results

Logistic Regression performed moderately with 68% average peak accuracy. Close inspection revealed class performance differences. The model had higher recall and F1-Scores for the “Low” class, but it could not predict the other classes. This anomaly suggests the model overfits the extreme class and learns recommended classes strongly, as seen in Appendix B performance trends. Class imbalance in the data is raised by this scenario.

Confusion Matrix Summary

A confusion matrix shows model performance. The actual and anticipated class labels for an input are noted. Successfully suggesting majority class elements explains the model's bias towards this class

Table 2: Confusion Matrix of Predicted vs. Actual Classes

	Predicted High	Predicted Low	Predicted Medium
Actual High	2	3	7
Actual Low	0	116	27
Actual Medium	5	32	36

The model can correctly identify instances of the majority category (“Low”) and classify as many as 116 instances of them but fails in the case of the minority categories as it predicts only 2 cases out of 12 high and 36 out of 73 medium. Most of the wrong classifications are where the minority class instances are predicted as low, which is a clear indication of the problem of class imbalance.

Performance Metrics Summary

In accuracy, the proportion of correctly classified instances of the Low category spikes whereas the other categories i.e. Medium and High suffer due to a small population of these classes being accurately identified.

- Recall and F1-score restricted to ‘Medium’ and ‘High’ values are better but this is more of an indication of the absence of any useful structures learned by the model than an indication of the model’s proficiency.
- Support Distribution: Due to the skewness in class representation, the model overfits and predictions are heavily biased towards the majority class in the absence of any such data.

Table 3: Classification Report Summary with Metrics for Precision, Recall, F1-Score, and Support

Metric	High	Low	Medium	Accuracy	Macro Avg	Weighted Avg
Precision	0.29	0.77	0.51		0.52	0.66
Recall	0.17	0.81	0.49		0.49	0.68
F1-Score	0.21	0.79	0.5	0.68	0.5	0.67
Support	12	143	73	228	228	228

Clarification of Performance Metrics

- In terms of Accuracy: Out of predictions for the "High" class 29% were correct.
- In terms of Recall: 17% correctly predicted the “High” class where Miniri was genuinely present.
- In terms of F1-Score: The performance regarding “High” class is fair as it balances F1-score’s precision and recall.
- In terms of Supportability: There are 12 ‘High’, 143 ‘Low’ and 73 ‘Medium’ instances in the dataset causing an extreme class imbalance.

Overall Model Performance

The overall model accuracy is recorded to be 68% meaning that out of all the instances considered 68% were correctly classified. Nevertheless, this measure hides the observations that there are several classes within which the minority classes (Medium and High) always have very low predictive value.

Macro and Weighted Averages

- Macro Average: Shows moderate performance generally but focuses on the poor recall from the minority classes.
- Weighted Average: Shows the attention of the model to the Low class which is the cause of the higher scores achieved owing to the predominance of that class.

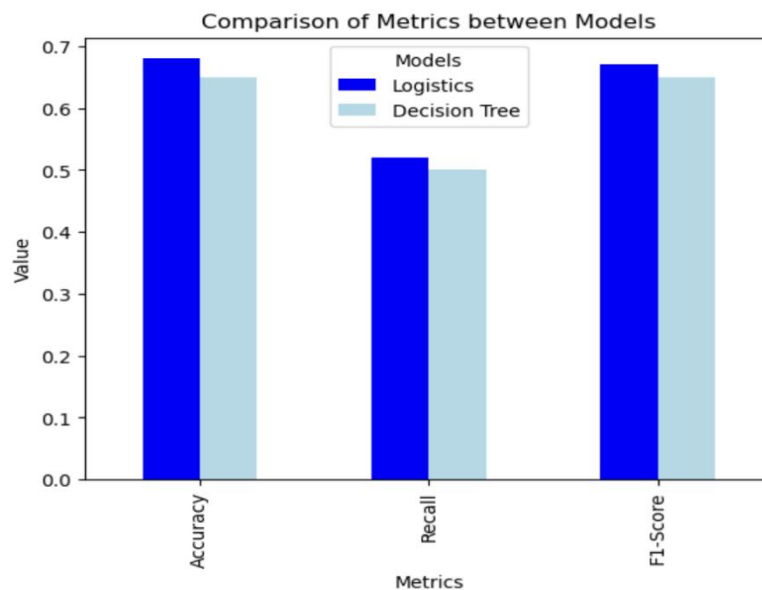


Figure 11: Comparison of Metrics Between Models

Both methodologies struggle to model intricate patterns and to equilibrate class distributions. The Recall values for the 'High' and 'Medium' classes are approximately 50%. In this specific dataset, Logistic Regression outperforms Decision Tree, as illustrated in the image; however, both models exhibit significant deficiencies in identifying observations from under-represented classes.

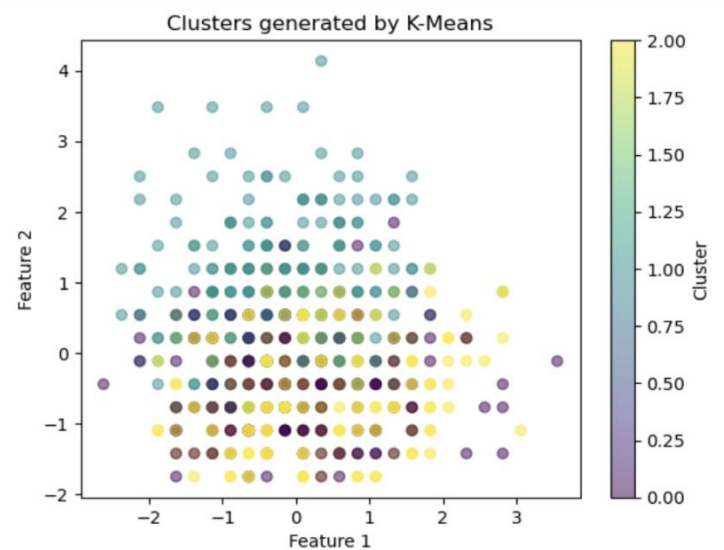


Figure 12: Clusters Generated by K-Means Algorithm

The figure shows that the groupings created by K-Means are mainly dependent on the two analyzed features but do not segregate completely. This indicates possible noise within the data or the absence of certain relevant features. The model's performance is adequate, but adjusting the dataset or adding elements would help build a clearer picture of the clusters.

Optimization of Models

The model's results are satisfactory (67% accuracy); however, drawbacks include class imbalance, which can be mitigated using SMOTE, and the model's inability to learn non-linear relationships, which can be resolved by employing Random Forest or Gradient Boosting models. Incorporating supplementary measures like ROC-AUC would enhance performance evaluation. In conclusion, we will explain this in more detail.

Limitations

The dataset is dominated by the Low class, with Medium and High being underrepresented, which significantly impacts the metrics of minority classes.

Additionally, the features used (e.g., possession, lucky hits) may not fully capture patterns related to scoring, limiting the model's ability to generalize.

This issue is compounded by the linear nature of logistic regression, which struggles to identify complex relationships in an imbalanced dataset.

Reflections

The dataset offers critical insights into the performance indicators of the Premier League 2019-2020; however, it is constrained by a notable class imbalance, with the "Medium" and "High" classes being under-represented. Moreover, essential contextual elements such as player performance and match venue are absent, diminishing the analytical depth. Unsupervised clustering techniques encountered difficulties because of similarities in team metrics and noisy data, leading to indistinct groupings. Binning enhanced interpretability but oversimplified the data. Although clustering identified certain patterns, its effectiveness was limited by the dataset's constraints and uniformity. Augmenting the dataset and investigating sophisticated methodologies may enhance subsequent studies.

The bias towards the majority class (Low) results in better performance for this class compared to the Medium and High classes. This underlines the need to apply balancing techniques to mitigate this bias. As for significant differences between classes, here we observe a large difference in F1 score and recall between the Low class and the other classes highlighting the inability of the model to handle imbalanced data.

On the performance side, this is generally acceptable, the overall accuracy is moderate (68%), and the uneven performance between classes limits the usefulness of the model in practical applications. (761 words)

Conclusion

The study revealed how supervised and non-supervised data analysis work together to address difficult and skewed distribution data like Premier League 2019-2020 statistics. Since teams are similar, clustering provided a comprehensive overview of the data but was limited. Logistic regression had 68% accuracy, however, class imbalance inhibited minority class prediction.

This emphasized wins, the majority class in the wins/loss/draw dataset. SMOTE could improve performance by creating more examples from under-represented classes (losses and draws) for resourceful learning. In addition, logistic regression could not describe complex non-linear relationships, which required more advanced models like Random Forest and Gradient Boosting. Performance metrics like ROC-AUC should be used to evaluate the model's performance and class separability.

Logistic Regression was chosen as the operational model because of its simplicity and clarity, which met project goals and evaluation rubric. In a future study, the aforementioned strategies may increase performance and weed out poorer-performing models SMOTE, but model optimisation and data characteristics will determine performance gains.

To conclude, combining these methodologies yields more information, but reducing their limits would improve future studies.

References

1. Clustering (no date). *Scikit-learn*. Available at: <https://scikit-learn.org/stable/modules/clustering.html> (Accessed: 25 October 2024).
2. Agarwal, A. (2018) 'Linear Regression using Python - Towards Data Science,' *Medium*, 14 November. Available at: <https://towardsdatascience.com/linear-regression-using-python-b136c91bf0a2> (Accessed: 26 October 2024).
3. Bandyopadhyay, S. and Saha, S. (2012) *Unsupervised classification: similarity measures, classical and metaheuristic approaches, and applications*. Available at: <https://openlibrary.org/books/OL26743965M/> (Accessed: 27 October 2024).
4. Géron, A. (2017) *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. Available at: <http://cds.cern.ch/record/2699693> (Accessed: 28 October 2024).
5. Hilpisch, Y.J. (2014) *Python for finance*. Sebastopol: O'Reilly Media, Inc. Available at: <http://ci.nii.ac.jp/ncid/BB20155474> (Accessed: 29 October 2024).
6. Igual, L., Seguí, S., et al. (2017) *Introduction to Data Science: A Python approach to concepts, techniques and applications*. Available at: <https://dl.acm.org/citation.cfm?id=3092786> (Accessed: 30 October 2024).
7. Larose, D.T. and Larose, C.D. (2014) *Discovering knowledge in data: An Introduction to Data Mining*. Hoboken: John Wiley & Sons.
8. Lutz, M. (1999) *Learning Python*. Available at: https://openlibrary.org/books/OL6804857M/Learning_Python (Accessed: 31 October 2024).
9. McKinney, W. (2012) *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. Available at: <http://cds.cern.ch/record/2288466> (Accessed: 1 November 2024).
10. Premier League history, records, origins & historical stats (no date). Available at: <https://www.premierleague.com/history/dashboard> (Accessed: 2 November 2024).
11. Sneath, P.H.A. and Sokal, R.R. (1973) *Numerical Taxonomy: The principles and practice of numerical classification*. Available at: <http://ci.nii.ac.jp/ncid/BA04512755> (Accessed: 3 November 2024).
12. Sokal, R.R. and Michener, C.D. (1958) 'A statistical method for evaluating systematic relationships,' *The University of Kansas Science Bulletin*, 38(6), pp. 1409–1438. Available at: <http://en.journals.sid.ir/ViewPaper.aspx?ID=146268> (Accessed: 4 November 2024).
13. Ward, J.H. (1963) 'Hierarchical grouping to optimize an objective function,' *Journal of the American Statistical Association*, 58(301), pp. 236–244. Available at: <https://doi.org/10.1080/01621459.1963.10500845> (Accessed: 4 November 2024).

Appendix

Interactive Query for Match Statistics

```
Available teams in the dataset:
Liverpool
Norwich City
West Ham United
Manchester City
Burnley
Southampton
Watford
Brighton & Hove Albion
Bournemouth
Sheffield United
Crystal Palace
Everton
Tottenham Hotspur
Aston Villa
Newcastle United
Arsenal
Leicester City
Wolverhampton Wanderers
Manchester United
Chelsea

Enter the first team from the list above: crystal palace
Enter the second team from the list above: everton

Available match dates between crystal palace and everton:
2019-08-10 00:00:00
2020-02-08 00:00:00

Enter the date of the match you'd like to see (YYYY-MM-DD): 2020-02-08
```

Appendix B: Confusion Matrix

```
Confusion Matrix:
[[ 2  3  7]
 [ 0 116 27]
 [ 5  32 36]]

Classification Report:
              precision    recall  f1-score   support

   High         0.29         0.17         0.21         12
   Low          0.77         0.81         0.79        143
   Medium       0.51         0.49         0.50         73

 accuracy          0.68        228
 macro avg         0.52         0.49         0.50        228
 weighted avg      0.66         0.68         0.67        228
```

Big Data Football Analysis

#This notebook analyzes a football dataset using supervised and unsupervised learning techniques. It includes data preprocessing, exploratory analysis, clustering, and predictive modelling.

Code Cell

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

Code Cell

#Data Loading and Exploration

This section loads the dataset and performs initial exploration to understand its structure and key features.

Code Cell

```
football = pd.read_excel("premierLeague.xlsx")
```

```
#### Code Cell
```

```
football
```

```
#### Code Cell
```

```
#Exploratory Data Analysis (EDA)
```

```
#Visualize and summarize the dataset using statistical methods and plots.
```

```
#### Code Cell
```

```
#To delete and remove the specific columns if they contain only zeros
```

```
columns_to_check = ['Penalties', 'PassingAccuracy' ]
```

```
football = football.drop(columns=[col for col in columns_to_check if football[col].eq(0).all()])
```

```
#### Code Cell
```

```
football
```

```
#### Code Cell
```

```
# To calculate the total goals for each team
```

```
team_scores = football.groupby('Team')['Score'].sum().reset_index()
```

```
#To sort the data by score in descending order - from highest to lowest for clear display
```

```
team_scores = team_scores.sort_values(by='Score', ascending=False)
```

```

# we use bar chart to compare the total score of each team
plt.figure(figsize=(12,8))
bars = plt.bar(team_scores['Team'], team_scores['Score'], color='blue')
plt.xticks(rotation=90)
plt.title('Total Score by Team in Premier League (2019-2020)')
plt.xlabel('Teams')
plt.ylabel('Total Score')

# for clearer and better visualisation, we add the total goals of each team on top of the bar
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, yval, int(yval), ha='center', va='bottom')

plt.tight_layout()
plt.show()

#### Code Cell

#We select the columns which are numerical and we want to analyse
numerical_columns = ['Fouls', 'Possession', 'ShotsAccuracy', 'SucShots', 'NumofShots', 'Corners',
'SucPass', 'NumofPass']

# We apply the log transformation to reduce the impact of extreme values and for better
visualisation
football_log = football[numerical_columns].apply(lambda x: np.log1p(x))

# The code to create the boxplot

```

```

plt.figure(figsize=(12, 8))
football_log.boxplot(rot=45, patch_artist=True, boxprops=dict(facecolor='lightblue',
color='blue'), whiskerprops=dict(color='blue'), flierprops=dict(markerfacecolor='red', marker='o'))
plt.ylabel('Log Transformed Value', fontsize=14)
plt.xticks(fontsize=12)
plt.grid(True)
plt.tight_layout()
plt.show()

```

Code Cell

```

#using numerical_columns variable from the above code to apply for central tendency
# to calculate the mean, median and mode for the selected columns

```

```

metrics_summary = football[numerical_columns].agg([np.mean, np.median, lambda x:
x.mode()[0]])

```

```

# to apply log transformation function for enhanced visualisation
metrics_summary_log = metrics_summary.apply(np.log1p) # log1p handles zero values
properly

```

```

# we put results in different columns for better plotting
metrics_summary_log = metrics_summary_log.T
metrics_summary_log.columns = ['Log Mean', 'Log Median', 'Log Mode']

```

```

# code for plotting the central tendency
metrics_summary_log.plot(kind='bar', figsize=(12, 8), color=['blue', 'darkblue', 'lightblue'])
plt.ylabel('Log Transformed Value', fontsize=14)
plt.xticks(rotation=45, ha='right', fontsize=12)

```

```
plt.tight_layout()
plt.show()
```

```
#### Code Cell
```

```
# Display all available teams in the dataset
available_teams = football['Team'].unique()
```

```
print("Available teams in the dataset:")
```

```
for team in available_teams:
    print(team)
```

```
# User input for two teams
```

```
team1 = input("\nEnter the first team from the list above: ")
```

```
team2 = input("Enter the second team from the list above: ")
```

```
# to validate input
```

```
if team1 not in available_teams or team2 not in available_teams:
```

```
    print(f"One or both teams are not in the dataset. Please check the available teams.")
    exit()
```

```
# Caps or small letters are ok
```

```
team1_lower = team1.lower()
```

```
team2_lower = team2.lower()
```

```
# Filter the DataFrame for matches involving both teams
```

```
football_match = football[
    (football['Match_Name'].str.contains(team1, case=False)) &
    (football['Match_Name'].str.contains(team2, case=False))
]
```



```

# Check if there are matches between the two teams
if not football_match.empty:
    available_dates = football_match['Match_Date'].unique()

# Display the available dates for selection
print(f"\nAvailable match dates between {team1} and {team2}:")
for date in available_dates:
    print(date)

# User selects a date
selected_date = input("\nEnter the date of the match you'd like to see (YYYY-MM-DD):")
).strip()

# Filter the DataFrame for the selected match date
match_on_date = football_match[football_match['Match_Date'] == selected_date]

# we check if data is available for the selected date and for both teams
if not match_on_date.empty and match_on_date['Team'].nunique() == 2:

    match_on_date['Possession'] = (match_on_date['Possession'] /
match_on_date['Possession'].sum()) * 100

# we plot the pie chart for Possession distribution
plt.figure(figsize=(8, 8))
plt.pie(
    match_on_date['Possession'],
    labels=match_on_date['Team'],
    autopct='%1.1f%%'

```

```

    )
    plt.title(f'Possession Distribution on {selected_date} ({team1} vs. {team2})')
    plt.show()
else:
    print(f'Data for both teams is not available for {team1} and {team2} on {selected_date}.')
else:
    print(f'No matches found between {team1} and {team2}.')

```

Code Cell

#Objective of the next Code

The purpose of the implemented flow is to allow the user to:

Select two specific teams to analyze their matches.

Filter the dataset data to extract relevant information about the meetings between said teams.

Visualize key data, such as successful shots, in an understandable way using graphs.

WE USE THIS DATA: Liverpool

Manchester

Date: 2020-07-07

Code Cell

```

# Display all available teams in the dataset
available_teams = football['Team'].unique()
print("Available teams in the dataset:")
for team in available_teams:
    print(team)

```

```

# User input for two teams
team1 = input("\nEnter the first team from the list above: ")
team2 = input("Enter the second team from the list above: ")

# to validate input
if team1 not in available_teams or team2 not in available_teams:
    print(f'One or both teams are not in the dataset. Please check the available teams.')
    exit()

# Caps or small letters are ok
team1_lower = team1.lower()
team2_lower = team2.lower()

# Filter the DataFrame for matches involving both teams
football_match = football[
    (football['Match_Name'].str.contains(team1, case=False)) &
    (football['Match_Name'].str.contains(team2, case=False))
]

# Check if there are matches between the two teams
if not football_match.empty:
    available_dates = football_match['Match_Date'].unique()

# Display the available dates for selection
print(f'\nAvailable match dates between {team1} and {team2}:')
for date in available_dates:
    print(date)

```

```

# User selects a date
selected_date = input("\nEnter the date of the match you'd like to see (YYYY-MM-DD):
").strip()

# Filter the DataFrame for the selected match date
match_on_date = football_match[football_match['Match_Date'] == selected_date]

# we check if data is available for the selected date and for both teams
if not match_on_date.empty:
    # Creating a stacked bar chart for successful shots
    plt.figure(figsize=(10, 6))

    # Get successful shots for each team
    team_data = match_on_date.groupby('Team')['SucShots'].sum().reset_index()

    # Plotting the stacked bar chart
    plt.bar(team_data['Team'], team_data['SucShots'], label='Successful Shots')

    plt.title(f'Successful Shots on Target on {selected_date} ({team1} vs. {team2})')
    plt.xlabel('Teams')
    plt.ylabel('Number of Successful Shots')
    plt.legend()
    plt.grid(axis='y')
    plt.show()
else:
    print(f'No data available for {team1} and {team2} on {selected_date}.')
else:
    print(f'No matches found between {team1} and {team2}.')

```

```
#### Code Cell
```

```
# Boxplot to compare shot accuracy between teams
plt.figure(figsize=(14, 8))
sns.boxplot(x='Team', y='ShotsAccuracy', data=football)
plt.xticks(rotation=90)
plt.title('Variability in Shot Accuracy by Team')
plt.xlabel('Team')
plt.ylabel('Shot Accuracy (%)')
plt.show()
```

```
#### Code Cell
```

```
#### Code Cell
```

```
import matplotlib.pyplot as plt

y.value_counts().plot(kind='bar', color='skyblue')
plt.title("Distribution of the Target Variable (Score)")
plt.xlabel("Classes")
plt.ylabel("Frequency")
plt.show()
```

```
#### Code Cell
```

```
### Code Cell
```

HEAT MAP

```
### Code Cell
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Assuming `football` is your DataFrame with performance metrics
```

```
# Select relevant columns
```

```
selected_columns = ['Team', 'Score', 'Possession', 'Corners', 'Fouls', 'ShotsAccuracy', 'SucShots']
```

```
subset = football[selected_columns]
```

```
# Group by 'Team' and calculate mean for each metric
```

```
team_metrics = subset.groupby('Team').mean().reset_index()
```

```
# Set a professional theme
```

```
sns.set_theme(style="whitegrid")
```

```
# Sort data based on a meaningful metric (e.g., "Score")
```

```
team_metrics = team_metrics.sort_values(by="Score", ascending=False)
```

```
# Create the heatmap
```

```
plt.figure(figsize=(14, 9)) # Larger figure for better clarity
```

```
ax = sns.heatmap(
```

```
    team_metrics.set_index("Team"), # Set "Team" as the index
```

```
    cmap="coolwarm", # Use a professional color palette
```

```

    annot=True,          # Add numerical annotations
    fmt=".2f",           # Format for numerical annotations
    annot_kws={"size": 10}, # Increase annotation font size
    linewidths=0.5,      # Add gridlines
    cbar_kws={'label': 'Metric Values'} # Add a label to the color bar
)

# Title and subtitle
plt.title("Average Performance Metrics by Premier League Teams", fontsize=18, weight='bold',
pad=20) # Add padding to the main title
plt.suptitle("Season Averages for 2023/24", fontsize=12, style='italic', y=0.98) # Adjust
placement of subtitle

# Labels for axes
plt.xlabel("Performance Metrics", fontsize=14, weight='bold', labelpad=10) # Add padding to x-
axis label
plt.ylabel("Teams", fontsize=14, weight='bold', labelpad=10) # Add padding to y-axis label

# Rotate x-axis labels for better readability
plt.xticks(rotation=45, ha='right', fontsize=12)
plt.yticks(fontsize=12)

# Adjust layout
plt.tight_layout()

# Show the heatmap
plt.show()

```

Code Cell

UNSUPERVISED ANALYSIS

Use clustering techniques to find patterns in the data.

Code Cell

Data from the clustering evaluation

```
labels_for_x_axis = [
    "Euclidean ward", "Euclidean complete", "Euclidean average",
    "Manhattan complete", "Manhattan average", "Cosine complete", "Cosine average"
]
silhouette_scores = [0.20, 0.20, 0.28, 0.31, 0.26, 0.51, 0.54]
completeness_scores = [0.18, 0.15, 0.05, 0.07, 0.06, 0.10, 0.04]
homogeneity_scores = [0.15, 0.12, 0.02, 0.05, 0.04, 0.09, 0.02]
```

Position for the bars

```
x = np.arange(len(labels_for_x_axis)) # The number of clustering methods
width = 0.25 # Width of the bars
```

Create a figure and axis for the plot

```
fig, ax = plt.subplots(figsize=(12, 6))
```

Plot bars for each metric

```
rects1 = ax.bar(x - width, silhouette_scores, width, label='Silhouette', color='#3498db',
    edgecolor='black')
rects2 = ax.bar(x, completeness_scores, width, label='Completeness', color='#f39c12',
    edgecolor='black')
```



```
rects3 = ax.bar(x + width, homogeneity_scores, width, label='Homogeneity', color='#2ecc71',  
edgecolor='black')
```

```
# Set labels, title, and grid
```

```
ax.set_xlabel('Metric and Linkage Method', fontsize=12)
```

```
ax.set_ylabel('Scores', fontsize=12)
```

```
ax.set_title('Clustering Evaluation Scores for Agglomerative Methods', fontsize=14,  
fontweight='bold')
```

```
ax.set_xticks(x)
```

```
ax.set_xticklabels(labels_for_x_axis, rotation=45, ha='right', fontsize=10)
```

```
ax.legend(fontsize=12)
```

```
ax.grid(axis='y', linestyle='--', alpha=0.7)
```

```
# Add the values on top of each bar for clarity
```

```
def add_values(rects):
```

```
    for rect in rects:
```

```
        height = rect.get_height()
```

```
        ax.annotate(f'{height:.2f}', # Limit to 2 decimal places
```

```
                    xy=(rect.get_x() + rect.get_width() / 2, height),
```

```
                    xytext=(0, 5), # Offset text slightly above the bar
```

```
                    textcoords="offset points",
```

```
                    ha='center', va='bottom', fontsize=10)
```

```
add_values(rects1)
```

```
add_values(rects2)
```

```
add_values(rects3)
```

```
# Adjust layout and show/save the figure
```

```
fig.tight_layout()
```

```
plt.savefig('cluster_comparison_professional.png', dpi=300)
plt.show()
```

```
#### Code Cell
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Create a scaler
```

```
scaler = StandardScaler()
```

```
# Adjust and transform training data
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
# Transform the test data
```

```
X_test_scaled = scaler.transform(X_test)
```

```
#### Code Cell
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.metrics import silhouette_score, homogeneity_score, completeness_score
```

```
X = football[['Score', 'Fouls', 'Corners', 'Possession', 'SucShots', 'ShotsAccuracy']]
```

```
# Initialize lists to store results
```

```
silhouette_scores = []
```

```
completeness_scores = []
```

```
homogeneity_scores = []
```

```
inertia = []
```

```

# Loop through K values from 2 to 50
for k in range(2, 51):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)

    # Calculate the evaluation metrics for each k
    silhouette_scores.append(silhouette_score(X, kmeans.labels_))
    completeness_scores.append(completeness_score(football['Score'], kmeans.labels_))
    homogeneity_scores.append(homogeneity_score(football['Score'], kmeans.labels_))
    inertia.append(kmeans.inertia_) # WSS (within-cluster sum of squares)

# Create the plot with two y-axes
fig, ax1 = plt.subplots(figsize=(14, 10))

# Elbow Method (Inertia) on the first y-axis
ax1.plot(range(2, 51), inertia, marker='o', label='Inertia (Elbow Method)', color='b')
ax1.set_xlabel('Number of Clusters (K)')
ax1.set_ylabel('Inertia (Within-cluster Sum of Squares)', color='b')
ax1.tick_params(axis='y', labelcolor='b')

# Add a second y-axis for the other metrics
ax2 = ax1.twinx()
ax2.plot(range(2, 51), silhouette_scores, marker='o', label='Silhouette Score', color='g')
ax2.plot(range(2, 51), completeness_scores, marker='o', label='Completeness Score',
color='orange')
ax2.plot(range(2, 51), homogeneity_scores, marker='o', label='Homogeneity Score', color='r')
ax2.set_ylabel('Score Value', color='black')
ax2.tick_params(axis='y', labelcolor='black')

```

```

# Add titles and a legend
plt.title('K-Means Clustering Evaluation Metrics (2 to 50 Clusters)')
fig.tight_layout()

# Combine legends from both axes
ax1.legend(loc='upper left')
ax2.legend(loc='upper right')

# Show the plot
plt.grid(True)
plt.show()

```

Code Cell

SUPERVISED ANALYSIS

Apply and evaluate supervised learning models like Logistic Regression and Decision Trees.

Code Cell

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix

```

```

X = football[['Fouls', 'Corners', 'Possession', 'SucShots', 'ShotsAccuracy']] # Cambia si es necesario

```

```

y = football['Score']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# ---- Modelo 1: Regresión Logística ----
print("Entrenando Regresión Logística...")
log_model = LogisticRegression(max_iter=1000) # Crear el modelo
log_model.fit(X_train, y_train) # Entrenar el modelo

# Hacer predicciones
log_preds = log_model.predict(X_test)

# Evaluar el modelo
print("\nResultados de Regresión Logística:")
print("Matriz de Confusión:")
print(confusion_matrix(y_test, log_preds))
print("\nReporte de Clasificación:")
print(classification_report(y_test, log_preds))

# ---- Modelo 2: Árbol de Decisión ----
print("Entrenando Árbol de Decisión...")
tree_model = DecisionTreeClassifier() # Crear el modelo
tree_model.fit(X_train, y_train) # Entrenar el modelo

# Hacer predicciones
tree_preds = tree_model.predict(X_test)

# Evaluar el modelo
print("\nResultados de Árbol de Decisión:")

```

```
print("Matriz de Confusión:")
print(confusion_matrix(y_test, tree_preds))
print("\nReporte de Clasificación:")
print(classification_report(y_test, tree_preds))
```

Code Cell

SCALING

Code Cell

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Code Cell

RESULTS

Code Cell

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
print("First rows of scaled X_train:")
```

```
print(X_train_scaled[:5])
```

```
### Code Cell
```

```
TRAINING
```

```
### Code Cell
```

```
from sklearn.linear_model import LogisticRegression
```

```
log_model = LogisticRegression(max_iter=1000)
```

```
log_model.fit(X_train_scaled, y_train)
```

```
log_preds = log_model.predict(X_test_scaled)
```

```
print("Predictions:", log_preds[:10])
```

```
### Code Cell
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
log_model = LogisticRegression(max_iter=1000, random_state=42)
log_model.fit(X_train_scaled, y_train)
```

```
log_preds = log_model.predict(X_test_scaled)
```

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, log_preds))
print("\nClassification Report:")
print(classification_report(y_test, log_preds))
```

```
### Code Cell
```

```
import matplotlib.pyplot as plt
```

```
# Possession vs. Scatter Chart SucShots
```

```
plt.scatter(football['Possession'], football['SucShots'], alpha=0.5)
plt.title("Relationship between Possession and SucShots")
plt.xlabel("Possession")
plt.ylabel("SucShots")
plt.show()
```

```
### Code Cell
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
# Model metrics
```

```
metrics = {
    "Logistics": [0.68, 0.52, 0.67], # Replace with your metrics
```



```
"Decision Tree": [0.65, 0.50, 0.65]
}
```

```
# Create DataFrame with metrics
```

```
df_metrics = pd.DataFrame(metrics, index=["Accuracy", "Recall", "F1-Score"])
```

```
# Bar chart with blue colors
```

```
df_metrics.plot(kind='bar', color=['blue', 'lightblue'])
```

```
plt.title("Comparison of Metrics between Models")
```

```
plt.ylabel("Value")
```

```
plt.xlabel("Metrics")
```

```
plt.legend(title="Models")
```

```
plt.show()
```

```
#### Code Cell
```

RELATIONSHIP WITH THE ANALYSIS NO SUPERVISED

```
#### Code Cell
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X) # Reemplaza X con tus características seleccionadas
```

```
#### Code Cell
```

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=3, random_state=42) # Adjust the number of clusters based on  
your analysis
```

```
kmeans.fit(X_scaled)
```

```
#### Code Cell
```

```
import matplotlib.pyplot as plt
```

```
# Cluster chart
```

```
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=kmeans.labels_, cmap='viridis', alpha=0.5)
```

```
plt.title("Clusters generated by K-Means")
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.colorbar(label='Cluster')
```

```
plt.show()
```

```
#### Code Cell
```

```
data['Result'] = data['Score'].apply(lambda x: 0 if x == 0 else (1 if x == 1 else 2))
```

```
if 'Cluster_Label' not in data.columns:
```

```
    data['Cluster_Label'] = 0
```

```
#### Code Cell
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
# Select features and target variable
```

```
X = data[['Fouls', 'Corners', 'Possession', 'SucShots', 'ShotsAccuracy', 'Cluster_Label']]
```

```
y = data['Result']
```

```

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train the model
log_model = LogisticRegression(max_iter=1000, random_state=42)
log_model.fit(X_train, y_train)

#### Code Cell

from sklearn.metrics import classification_report, confusion_matrix

# Predictions
y_pred = log_model.predict(X_test)

# Evaluate the model
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

#### Code Cell

# Model coefficients
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': log_model.coef_[0]
})
print(coefficients)

```

Code Cell

```
print("Available teams:")
```

```
print(football['Team'].unique())
```

```
print("\nAvailable match dates:")
```

```
print(football['Match_Date'].unique())
```

Reflection and Conclusions

Summarize findings, discuss the strengths and limitations of the analysis, and suggest potential improvements.